

Overview of Interrupt Handling and DMA Design

I/O Management and Data Transfer

Prashant Jadav

DEPSTAR - CHARUSAT

Why Do We Need Interrupts?

- A CPU executes instructions much faster than many I/O devices.
- Continuously checking a device is called **polling**.
- Polling wastes CPU cycles when the device is not ready.
- An **interrupt** allows a device or hardware component to request CPU attention.

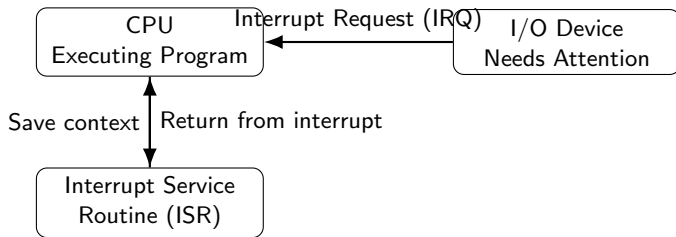
Basic idea

Instead of asking the device repeatedly “Are you ready?”, the CPU continues useful work and the device signals the CPU when attention is required.

Example

When a keyboard key is pressed, the keyboard controller can generate an interrupt rather than requiring the CPU to continuously check the keyboard.

Interrupt: Basic Concept



- The CPU temporarily suspends its current execution.
- The processor saves sufficient context to resume later.
- Control transfers to an **Interrupt Service Routine (ISR)**.
- After servicing the event, execution resumes from the interrupted program.

Interrupt Handling Sequence

- ➊ **Interrupt request:** A device or hardware component raises an interrupt.
- ➋ **Recognition:** CPU detects an enabled interrupt.
- ➌ **Context saving:** Processor saves required state such as program counter and status information.
- ➍ **Identify source:** Interrupt mechanism determines which ISR should execute.
- ➎ **ISR execution:** Operating-system/device-specific code services the event.
- ➏ **Context restoration:** Saved processor state is restored.
- ➐ **Return:** CPU resumes the interrupted execution.

Key principle

The ISR should perform the time-critical work quickly and defer lengthy processing when possible.

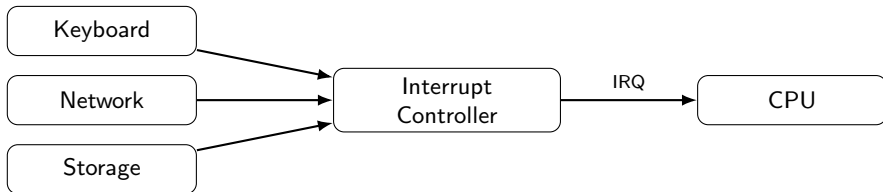
Interrupt Vector and Interrupt Controller

Interrupt Vector

- A mechanism for locating the appropriate ISR.
- An interrupt number/vector identifies the service routine.
- The processor uses the vector to obtain the ISR address.

Interrupt Controller

- Collects interrupt requests from multiple devices.
- Masks or enables selected interrupts.
- Can prioritize simultaneous requests.
- Delivers an interrupt request to the CPU.



Types of Interrupts

Hardware interrupt Generated by hardware such as keyboard, timer, network interface, or storage controller.

Software interrupt Explicitly generated by software to request a service or transfer control.

Maskable interrupt Can be temporarily disabled or masked by the processor.

Non-maskable interrupt (NMI) Used for high-priority events that normally cannot be ignored.

Timer interrupt Periodic interrupt used by the OS for scheduling and time management.

Operating-system example

A periodic timer interrupt allows the OS scheduler to regain control and implement preemptive multitasking.

Polling vs. Interrupt-Driven I/O

Aspect	Polling	Interrupt-driven I/O
CPU behavior	Repeatedly checks device status	Continues other work until interrupted
CPU utilization	Can waste CPU cycles	Generally more efficient for infrequent events
Response	Depends on polling interval	Event can trigger immediate attention
Implementation	Relatively simple	Requires interrupt mechanism and ISR
Typical use	Very simple/regular hardware interaction	General-purpose I/O systems

Important

Interrupts reduce unnecessary CPU polling, but interrupt handling itself has overhead. Extremely high interrupt rates can overload the CPU.

Why DMA Is Needed

- Interrupt-driven I/O is useful for events, but large data transfers can generate many interrupts.
- If the CPU copies every byte/word between a device and memory, significant CPU time is consumed.
- **Direct Memory Access (DMA)** allows a DMA controller to transfer blocks of data directly between an I/O device and main memory.

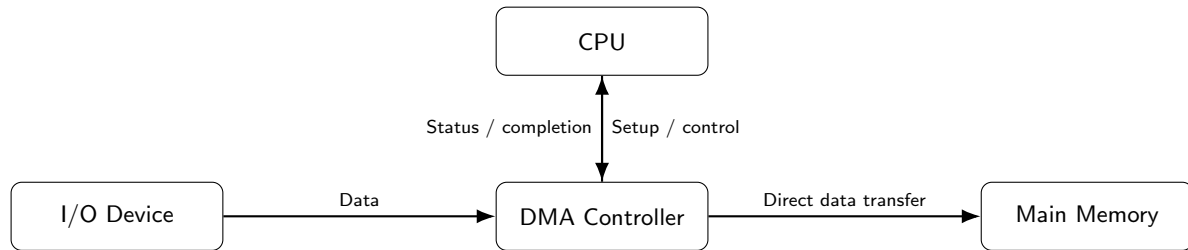
Central idea

CPU initializes the transfer; DMA controller performs the bulk data movement.

Example

Reading a large file from a storage device into RAM can be performed using DMA so the CPU does not have to copy every data word itself.

DMA Architecture



- CPU provides source, destination, direction, and transfer count.
- DMA controller manages the data movement.
- CPU can execute other work during much of the transfer.
- DMA can generate an interrupt when the transfer completes.

DMA Transfer: Step-by-Step

- ① CPU configures the DMA controller.
- ② DMA controller receives the starting memory address.
- ③ CPU specifies transfer direction and number of bytes/words.
- ④ DMA controller requests access to the system bus.
- ⑤ Data moves directly between the I/O device and memory.
- ⑥ DMA updates the address and transfer count after each transfer.
- ⑦ When the block is complete, DMA signals completion, typically using an interrupt.

CPU involvement

Before: configure DMA

During: mostly free for other work

After: handle completion

Common DMA Transfer Modes

Block / Burst mode DMA transfers a block of data while retaining control of the bus.

Cycle stealing DMA transfers one word/byte at a time by temporarily taking bus cycles from the CPU.

Transparent DMA DMA uses the bus when the CPU is not using it, minimizing interference with CPU execution.

Trade-off

More aggressive DMA access can increase transfer throughput but may reduce CPU memory/bus availability.

Interrupt-Driven I/O vs. DMA

Feature	Interrupt-driven I/O	DMA
CPU data movement	CPU participates in transfer	DMA controller moves bulk data
Best suited for	Events and smaller transfers	Large/block transfers
CPU overhead	Higher for frequent transfers	Lower for bulk data movement
Completion notification	Interrupt often used	Interrupt commonly signals completion
Hardware complexity	Lower	Higher
Example	Keyboard input	Disk/network block transfer

Design insight

DMA does not eliminate interrupts. A common design is: **DMA performs the transfer, then interrupts the CPU when the operation completes.**

Practical Example: Network Packet Reception

- ① Network interface receives a packet.
- ② The device uses DMA to place packet data into a memory buffer.
- ③ The device records completion/status information.
- ④ An interrupt informs the CPU that processing is required.
- ⑤ The OS network stack processes the packet.
- ⑥ The buffer is eventually returned to the device for reuse.

Why this design?

Using DMA for packet data avoids requiring the CPU to copy every byte from the network device, while the interrupt provides notification that useful work is ready.

Design Considerations

- **Latency:** How quickly must the system respond to an event?
- **Throughput:** How much data must be transferred per second?
- **CPU overhead:** How much processor time is consumed by I/O?
- **Bus contention:** How does DMA affect CPU access to memory?
- **Buffer management:** Where is data stored before and after processing?
- **Synchronization:** How are CPU, device, and DMA kept consistent?
- **Reliability:** What happens when a transfer fails or an interrupt is lost?

OS design perspective

Efficient I/O requires balancing CPU utilization, response time, memory bandwidth, device throughput, and implementation complexity.

Key Takeaways

- An **interrupt** provides asynchronous notification that CPU attention is required.
- An **ISR** contains the code that handles an interrupt event.
- An **interrupt controller** manages multiple interrupt sources and priorities.
- **Polling** repeatedly checks device status; interrupts allow event-driven notification.
- **DMA** transfers blocks of data directly between devices and main memory with limited CPU involvement.
- DMA commonly uses an interrupt to notify the CPU after completing a transfer.
- Good I/O design balances **latency, throughput, CPU overhead, and bus utilization**.

Interrupts notify; DMA moves data efficiently.

Quick Check / Discussion

- ① Why is polling inefficient when a device produces events infrequently?
- ② What information must the CPU provide when configuring a DMA transfer?
- ③ Why can a very high interrupt rate reduce system performance?
- ④ How does DMA reduce CPU involvement in large data transfers?
- ⑤ Why is an interrupt still useful after a DMA transfer?

Think

For a high-speed network interface receiving thousands of packets per second, would generating one CPU interrupt for every packet always be desirable? Explain the performance trade-off.